

CS 230 Project Proposal

Zachary Snyder, Hamed Gorjiara, Seungmok Lee

January 27, 2017

Static Analysis

What happens when the load balancer/scheduler has knowledge of the structure of the programs that it is scheduling?

For example, there is a language/runtime system known as Cilk. The current implementation of this is Cilk Plus, as produced by Intel. What's interesting about Cilk is that the runtime can make hard guarantees about the performance of the program. The runtime guarantees that the total running time of any job, the total memory footprint at any given time, and the total communication overhead is all within a constant factor of optimum. It seems like having these guarantees about multiple jobs in a distributed system would effectively solve the happiness equation no matter what the users throw at you.

The caveat here is that Cilk requires that you write your code in the Cilk language. This allows the compiler to statically analyze the dependencies between the different possible processes in the program, and pass that information to the runtime.

Can these properties be generalized to a distributed system with multiple jobs? What can you do if you have more information about your program than if you assume that it is a black box?

Dynamic Processes

What happens when jobs are dynamically creating and destroying processes? In the context of the model we've been discussing in class, we can view this as a state change where we recompute, but what if we knew things about the dependency structure of these new processes? We could more accurately predict whether or not it is a good idea to migrate these new processes to other cores, or even which cores they should migrate to.